

Ensemble techniques

Ensemble techniques in machine learning combine multiple models (often called "weak learners") to produce a single, superior predictive model with improved accuracy, reduced variance, and better generalization.

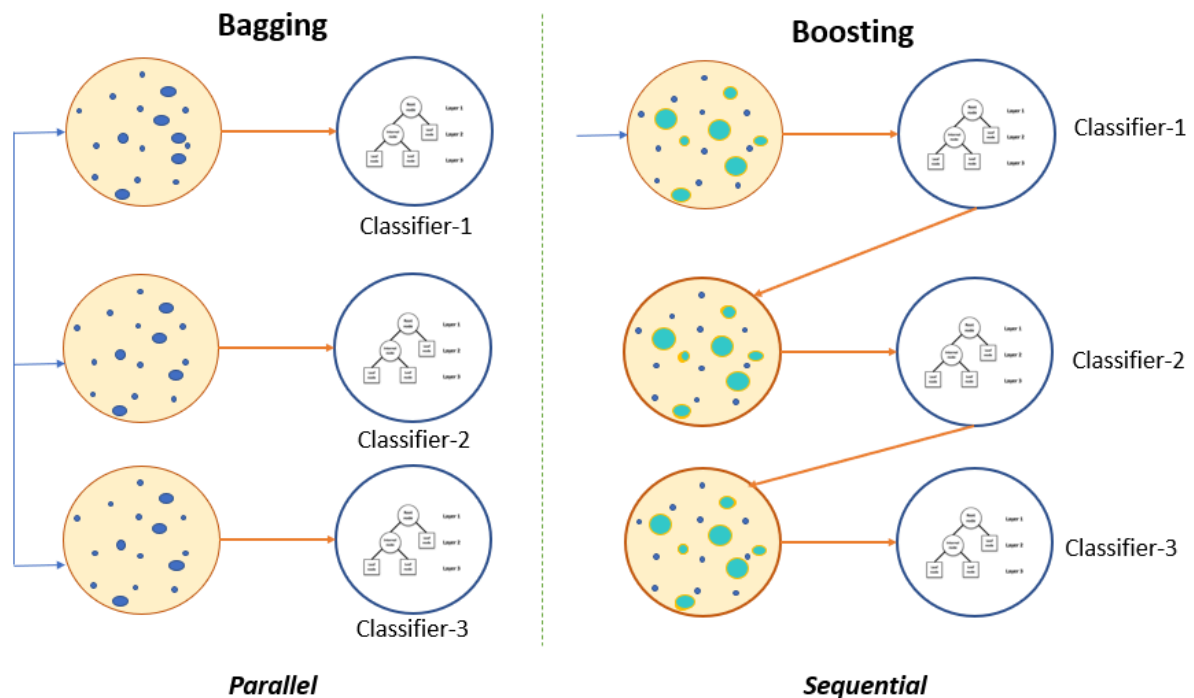
Bagging and boosting

Bagging (Bootstrap Aggregation): Trains multiple models in parallel on random, independent subsets of the training data. The final prediction is an average (regression) or majority vote (classification).

- *Example:* Random Forest uses multiple decision trees to reduce overfitting.

Boosting: Trains models sequentially, where each new model attempts to correct the errors (misclassified data points) of the previous model.

- *Example:* AdaBoost, Gradient Boosting, XGBoost.



- Stacking (Stacked Generalization): Combines different types of models (e.g., Logistic Regression, Decision Tree) by training a meta-model to determine the best combination of their predictions.
- Voting: Combines outputs from various models, often used in classification to select the majority-voted class.

Random Forest and XGBoost are two highly popular and powerful ensemble learning algorithms, particularly effective for classification and regression tasks. While both leverage decision trees, they do so with fundamentally different strategies: **bagging** for Random Forest and **boosting** for XGBoost.

Random Forest Classifier

Random Forest is an ensemble learning method based on bagging (Bootstrap Aggregating). It builds a "forest" of multiple decision trees and combines their predictions to produce a more accurate and stable result than a single decision tree.

From the original training dataset, Random Forest creates multiple (e.g., 100 or 500) random subsets of the data with **replacement**. This means some data points may appear multiple times in a subset, and some may not appear at all (these are called "out-of-bag" samples and can be used for internal validation).

Each subset is used to train a separate decision tree.

Feature Randomness:

When building each individual decision tree in the forest, at each split node, the algorithm doesn't consider all available features. Instead, it randomly selects a **subset of features** (e.g., square root of the total number of features) from which to choose the best split. This further decorrelates the trees.

The Classification vs. Regression Distinction

It is important to note that the $\sqrt{p}$ rule is most commonly associated with **classification** tasks. [🔗](#)

For **regression** tasks, a different heuristic is typically used:

- **Classification:** $\sqrt{p}$ features (e.g., if you have 100 features, you might try 10 at each split).
- **Regression:** $p/3$ features (e.g., if you have 100 features, you might try about 33 at each split) [2]. [🔗](#)

Tree Construction:

Each decision tree is grown to its full depth without pruning (or with minimal pruning). The randomness introduced by bootstrap sampling and feature subsetting helps prevent overfitting, even with deep trees.

Prediction (Voting):

For a new data point, each individual decision tree in the forest makes a prediction (a "vote" for a class).

The final prediction of the Random Forest classifier is determined by **majority voting** among all the trees. For example, if 70 out of 100 trees predict "Class A" and 30 predict "Class B," the Random Forest predicts "Class A."

XGBoost Classifier (eXtreme Gradient Boosting)

XGBoost is an advanced and highly optimized implementation of gradient boosting. It's an ensemble learning method where new models are added sequentially to correct the errors made by previous models. It's known for its speed, performance, and ability to handle large datasets efficiently.

Sequential Building (Boosting):

XGBoost starts with an initial "base" prediction (often a simple constant, like 0.5 for classification).

It then builds the first decision tree (a "weak learner") to predict the residuals (the errors) of the initial prediction.

Subsequent trees are built sequentially, with each new tree trying to correct the errors (residuals) of the *combined* predictions of all previously built trees. This iterative error correction is the core of boosting.

Gradient Descent Optimization:

Instead of simply fitting to residuals, XGBoost uses gradient descent to minimize a specified loss function (e.g., logistic loss for classification) by iteratively moving in the direction of the steepest descent of the error.

Regularization:

A key feature of XGBoost is its built-in regularization techniques (L1 and L2 regularization) on the tree complexities. This helps prevent overfitting by penalizing large weights and complex trees, leading to more generalized models.

Tree Pruning and Optimization:

XGBoost builds trees in a level-wise (breadth-first) manner rather than depth-first. This allows for parallel computation of splits within a level.

It also incorporates a "pruning" mechanism based on a "gain" threshold. It calculates the gain for each potential split and only keeps splits that significantly improve the model's objective function. This prevents trees from growing too deep and overfitting.

Handling Missing Values:

XGBoost has an intelligent way of handling missing values. During training, it learns the best direction to classify data points with missing values (left or right split) by trying both options and seeing which one minimizes the loss.

Prediction (Weighted Sum):

The final prediction for a new data point is the weighted sum of the predictions from all individual trees in the ensemble.

Each tree's contribution is scaled by a learning rate (shrinkage parameter).